

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Containerization with Docker Data _____
Class Applied Computing Name Lu Moyou ID 202483910013

一、Experimental purpose

Learn how to package a web application into a Docker Image.

Understand the concept of Immutable Infrastructure.

Practice mapping network ports between a Cloud Container and the Host.

二、Experiment Contents

We will now build a Docker image for your previous index.html

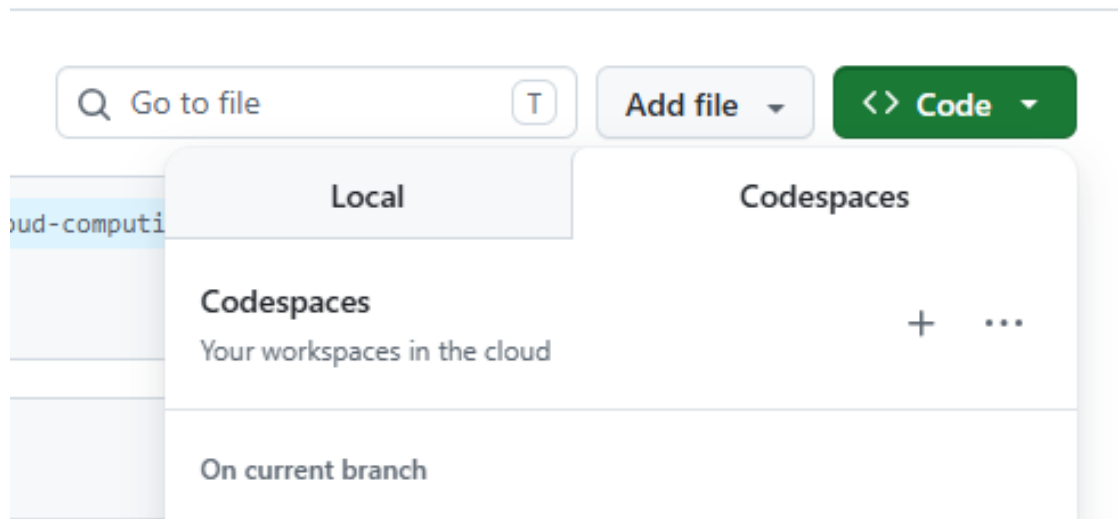
三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage.

Screenshots must demonstrate that they were taken on your own computer, for example by including visible system time, desktop environment, or other identifiable personal elements.

✓ Environment Setup

1. Open your GitHub repository.
2. Click the Code button and select Codespaces.



3. Click Create codespace.

Step 1: Verify Docker Environment

In your Codespaces terminal (at the bottom), type the following command to ensure the Docker daemon is running:

Bash: `docker version`

Observation: You should see both Client and Server version information.

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker version
Client:
 Version:           29.3.0-1
 API version:       1.54
 Go version:        go1.26.0
 Git commit:        5927d80c76b3ce5cf782be818922966e8a0d87a3
 Built:            Fri Mar  6 18:50:47 2026
 OS/Arch:          linux/amd64
 Context:          default

Server:
 Engine:
  Version:          29.3.0-1
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.0
  Git commit:       83bca512aa7ffc1bb4f37ce1107e0d3e3489ad43
  Built:           Fri Mar  6 18:29:13 2026
  OS/Arch:         linux/amd64
  Experimental:     false
 containerd:
  Version:         2.2.1-1
  GitCommit:       dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:        1.4.0-2
  GitCommit:      8bd78a9977e604c4d5f67a7415d7b8b8c109cdc4
 docker-init:
  Version:        0.19.0
  GitCommit:
```

Step 2: Create a Dockerfile

A Dockerfile is a script containing instructions on how to build your image.

In the file explorer on the left, click New File.

Name it exactly Dockerfile (case - sensitive, no file extension).

Paste the following code:

```
# Use Nginx (a popular web server) as the base image
```

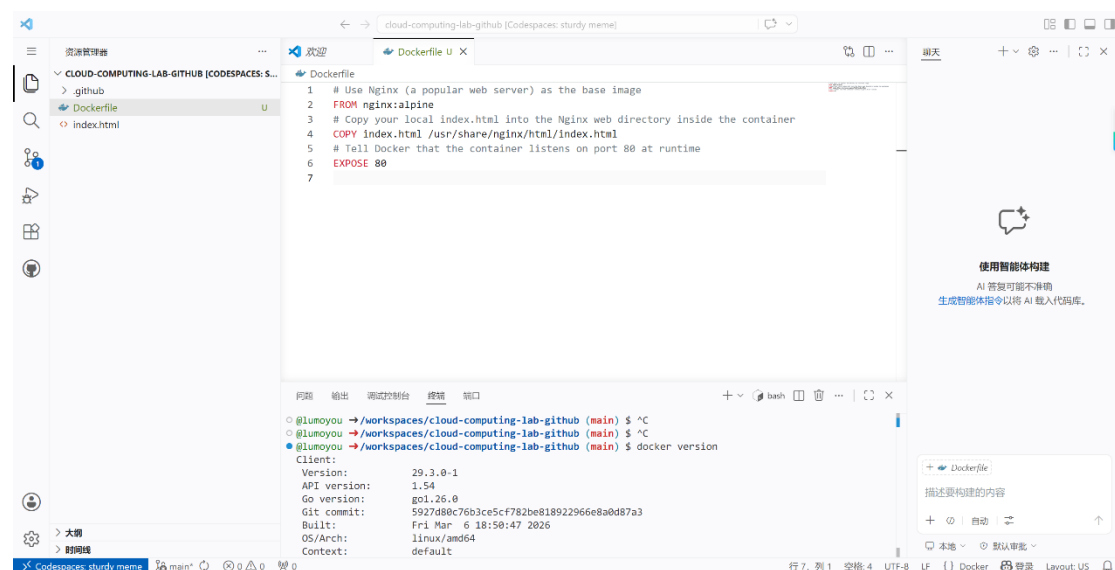
```
FROM nginx:alpine
```

```
# Copy your local index.html into the Nginx web directory  
inside the container
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
# Tell Docker that the container listens on port 80 at runtime
```

```
EXPOSE 80
```



Step 3: Build the Docker Image

Run the following command to "bake" your code into a reusable image:

Bash: `docker build -t cloud-lab-image:v1 .`

Cloud Concept: The `.` at the end tells Docker to look for the Dockerfile in the current folder.

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 311B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  2.3s
=> [auth] library/nginx:pull token for registry-1.docker.io                    0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load build context                                                 0.0s
=> => transferring context: 904B                                                0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd22 2.2s
=> => resolve docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd22 0.0s
=> => sha256:94de7c34b8d25a6e2241d9f9af135eba064193ceca1b6b782e80d9d8738c24e8 20.28MB / 20.28MB 0.6s
=> => sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 1.39kB / 1.39kB 0.7s
=> => sha256:86e0029eccff957b151ddd0fb727628e9ae5fcde85cb9771e74d471efe031245 1.21kB / 1.21kB 0.7s
=> => sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 400B / 400B 0.7s
=> => sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 951B / 951B 0.3s
=> => sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb9c92dec27d9f2dcca45a2 625B / 625B 0.3s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.4s
=> => sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 1.88MB / 1.88MB 0.3s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.2s
=> => extracting sha256:16e661a44239bffc66a00e3a0d6c45af7059317a60a5685760937a68d0b3b624 0.2s
=> => extracting sha256:2c084cd9252c2e4b305bd605f0b6534943993f25adfb9c92dec27d9f2dcca45a2 0.0s
=> => extracting sha256:04a7ff7b108a8f7d67888aa7f32283bcda09520660a1d149bd4e6a294cf8b135 0.0s
=> => extracting sha256:bcace0045394f92e9a583eb4f11a674f91cc91d407b85d52afe3f6209672ca1e 0.0s
=> => extracting sha256:86e0029eccff957b151ddd0fb727628e9ae5fcde85cb9771e74d471efe031245 0.0s
=> => extracting sha256:0faf356544319b35bb9aa03595e556e22277228565e7470fa7da4b1a2c999aa5 0.0s
=> => extracting sha256:94de7c34b8d25a6e2241d9f9af135eba064193ceca1b6b782e80d9d8738c24e8 0.5s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html                    1.0s
=> exporting to image                                                            0.2s
=> => exporting layers                                                            0.1s
=> => exporting manifest sha256:7bce2df7e68660f0553f97660ee3e5d41afac9bf029f0bbec2222e03a6b71e7 0.0s
=> => exporting config sha256:1cb781c26e699063b74f56b7c18e2967033cf5aee30df539e13c299d55cc4ab8 0.0s
=> => exporting attestation manifest sha256:864db2c488d6b901e1ebc7108c9777f21bc11deac4be4cc0fffc 0.0s
=> => exporting manifest list sha256:719dab814e5cc91a0ecb4926d6963222a91b2b29b078fa6c61e94be2af 0.0s
=> => naming to docker.io/library/cloud-lab-image:v1                          0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v1                      0.0s
```

Step 4: Run the Container

Now, launch a "Running Instance" (Container) based on your image:

Bash: `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1`

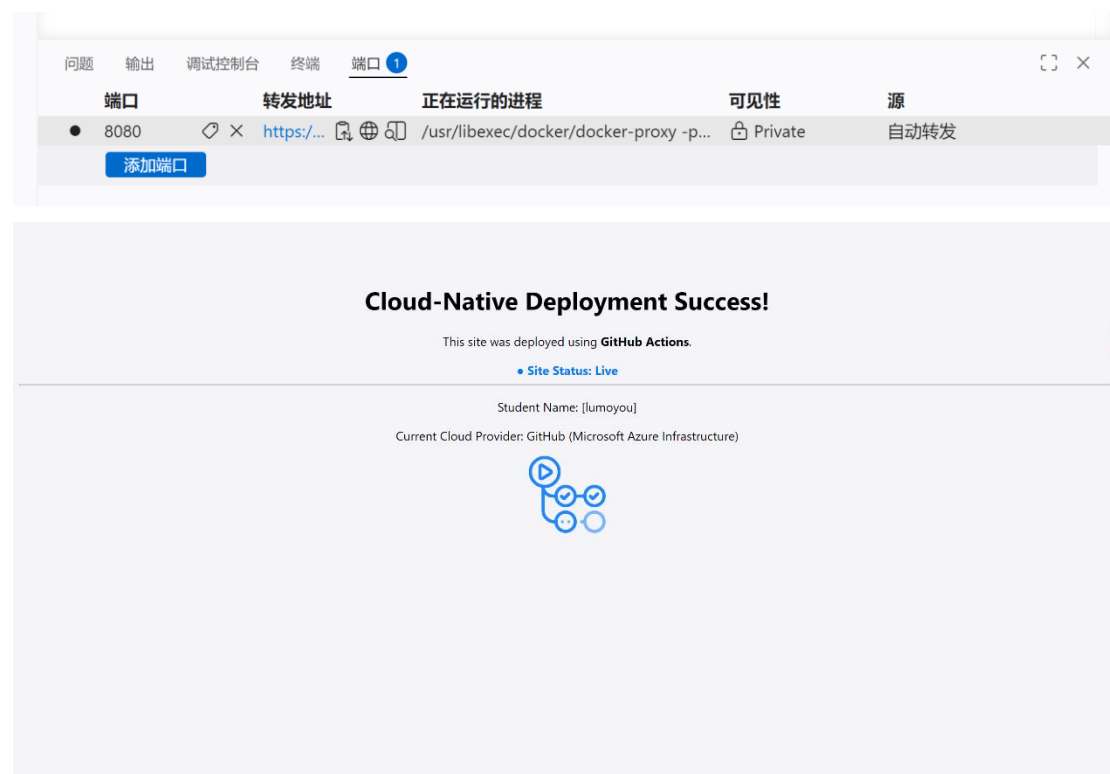
Explanation: `-p 8080:80` maps your Cloud VM's port 8080 to the Container's port 80.

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1  
c1bc3889e2053c14bf85eae7f606b0dff6af9f646bdd2d8dea8dfe84a1b5eafb
```

Step 5: Access the Web Server

Codespaces will detect a new port is active. A pop-up will appear in the bottom right: "A service is running on port 8080." Click Open in Browser.

Observation: You should see your personal `index.html` being served, but this time it is coming from inside a Docker container!



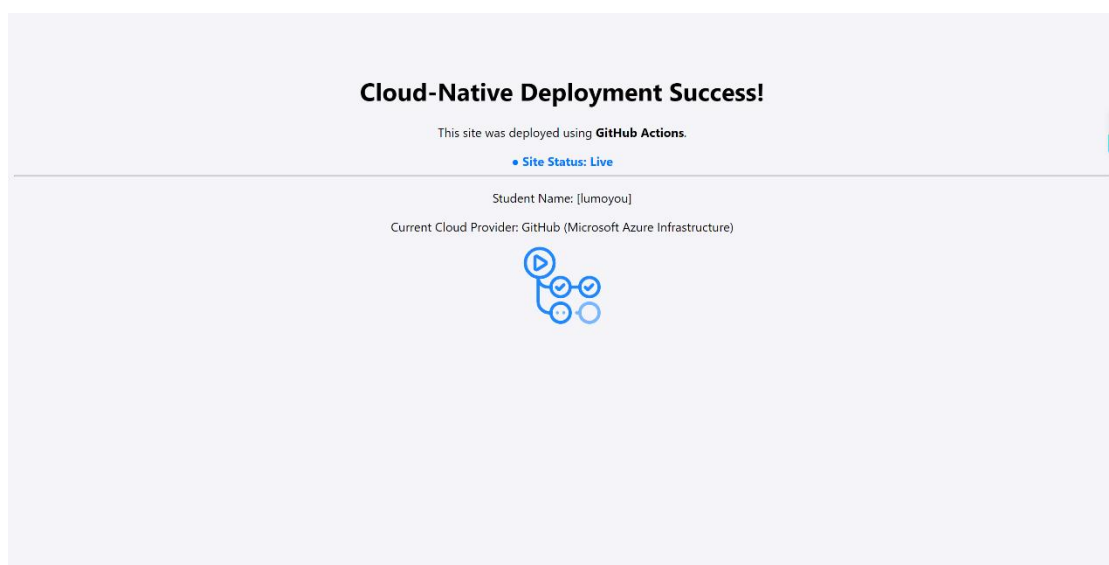
Step 6: Testing Container Immutability

In Cloud Computing, containers are immutable (unchangeable).

This experiment proves why we "build once and run anywhere."

- **Modify the Code:** Go to your index.html file and change the background color or the text (e.g., change "Success!" to "Updated!").
- **Refresh the Browser:** Go back to the tab where your site is running on port 8080 and refresh.
- **Observation:** The website did not change.
- **The Concept:** Even though you changed the source file, the Image running inside the container is a static snapshot of the past. To apply changes, you must rebuild the image and restart the container.

```
.status{ color: #007bff; font-weight: bold; } /* Change status to Blue */  
</style>  
  
{ color: #007bff; font-weight: red; } /
```



Step 7: Exploring the Container Interior (Shell Access)

A container is like a mini-computer. Let's "step inside" to see how the cloud organizes files.

Enter the Container: Run the following command to open a terminal inside your running container:

Bash: `docker exec -it my-web-container sh`

Navigate Files

Once the prompt changes (usually to a #), type these commands:

```
ls /usr/share/nginx/html/
```

```
cat /usr/share/nginx/html/index.html
```

Exit

Type `exit` to return to your main Cloud VM terminal.

The Concept

This demonstrates Isolation. The container has its own file system, separate from your Codespace.

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
/ # docker exec -it my-web-container sh
sh: docker: not found
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Cloud Computing Lab </title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      text-align: center;
      padding-top: 50px;
      background-color: #f4f4f9; /* Add a light grey background */
    }
    .status{ color: #007bff; font-weight: bold; } /* Change status to Blue */
  </style>
</head>
<body>
  <h1>Cloud-Native Deployment Success!</h1>
  <p>This site was deployed using <b>GitHub Actions</b></p>
  <div class="status">● Site Status: Live</div>
  <hr>
  <p>Student Name: [lumoyou]</p>
  <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
  
</body>
</html>
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ #
/ # exit
```

Step 8: Cloud Resource Monitoring

Cloud providers charge based on resource usage. Let's see how "lightweight" containers really are compared to traditional Virtual Machines.

1. Check Stats:

Run the command: `bash: docker stats`

2. Analyze:

Look at the MEM USAGE and CPU % columns in the real-time table.

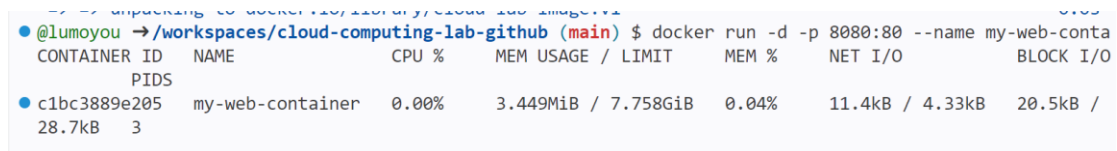
3. Observation:

Notice that a full web server is likely using less than 5MB of RAM. A traditional VM would require at least 512MB to 1GB to do the same task.

The Concept:

This is the Efficiency of the cloud. Containers share the host's OS kernel, allowing you to run hundreds of instances on a single server.

(Press Ctrl+C to stop the monitoring).



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
c1bc3889e205	my-web-container	0.00%	3.449MiB / 7.758GiB	0.04%	11.4kB / 4.33kB	20.5kB / 28.7kB

Step 9: Scaling and Port Management

In the cloud, "Scaling" means running multiple copies of the

same service to handle more traffic.

1. Run a Second Instance:

Launch another container using the same image but a different host port:
`docker run -d -p 8081:80 --name second-container cloud-lab-image:v1`

2. Verify:

Check the "Ports" tab in Codespaces. You should now have two active ports: 8080 and 8081. Open both to see identical sites.



端口	转发地址	正在运行的进程	可见性	源
8080	https://turbo-space-barnacl...	/usr/libexec/docker/docker-proxy -proto tcp -host-ip 0.0.0...	Private	自动转发
8081	https://turbo-space-barnacl...	/usr/libexec/docker/docker-proxy -proto tcp -host-ip 0.0.0...	Private	自动转发

3. Intentional Error:

Try running a third container using port 8080 again.

○ Observation:

Docker will throw an error: "Bind for 0.0.0.0:8080 failed: port is already allocated."

The Concept:

This is Scalability. We can spin up identical containers instantly, but we must manage the network traffic (ports) correctly.



端口	转发地址	正在运行的进程	可见性	源
8080	https://sturdy-m...	/usr/libexec/docker/docker-proxy -p...	Private	自动转发
8081	https://sturdy-m...	/usr/libexec/docker/docker-proxy -p...	Private	自动转发

添加端口

Step 10: Lab Cleanup

Professional cloud engineers always clean up resources to save costs and keep the environment tidy.

1. Stop and Remove: Stop the processes and delete the container instances:

```
Bash: docker stop my-web-container second-container  
docker rm my-web-container second-container
```

2. Verify Cleanup: Run the following command to ensure no containers are currently running: `bash: docker ps`

The Concept: In a "Pay-as-you-go" cloud model, stopping unused resources is essential to prevent unnecessary billing.

```
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker stop my-web-container second-container  
my-web-container  
second-container  
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker rm my-web-container second-container  
my-web-container  
second-container  
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Step 11: Use git to save your changes.

- `git add .` (Gather all your lab changes)
- `git commit -m "Finished Lab 02 "` (Seal the package)
- `git push` (Ship it to GitHub for permanent storage)

```
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ git add .  
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ git commit -m "Finished Lab 02 "  
[main 9f81f3d] Finished Lab 02  
1 file changed, 6 insertions(+)  
create mode 100644 Dockerfile  
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ git push  
To https://github.com/lumoyou/cloud-computing-lab-github  
! [rejected]        main -> main (fetch first)  
error: failed to push some refs to 'https://github.com/lumoyou/cloud-computing-lab-github'  
hint: Updates were rejected because the remote contains work that you do not  
hint: have locally. This is usually caused by another repository pushing to  
hint: the same ref. If you want to integrate the remote changes, use  
hint: 'git pull' before pushing again.  
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

There exists a problem,

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ git push --force origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 503 bytes | 503.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lumoyou/cloud-computing-lab-github
+ d78f35d...9f81f3d main -> main (forced update)
```

四、Reflection

1. Problems Encountered and Proposed Solutions

Problem 1: docker: permission denied when running Docker commands.

Solution: In Codespaces, this usually doesn't happen, but on a local machine, add your user to the docker group or use sudo. In Codespaces, ensure the terminal is fully initialized.

Problem 2: Port 8080 is already allocated when trying to run a third container.

Solution: Stop or remove the existing container using that port, or choose a different host port (e.g., -p 8082:80).

Problem 3: After building and running, the browser shows the Nginx default page instead of my index.html.

Solution: Check that index.html is in the same directory as the Dockerfile and that the COPY command in the Dockerfile has the correct paths (e.g., COPY index.html /usr/share/nginx/html/index.html).

Problem 4: docker build fails with "unable to prepare context:

unable to evaluate symlinks".

Solution: Make sure the Dockerfile is named exactly Dockerfile (no extension, case-sensitive) and that you are in the correct directory when running the command.

2. Explain why your website didn't update automatically after editing the code, and why this is actually a "feature" in cloud computing.

The website did not update automatically because **Docker containers are immutable**. When you build the image, the current version of index.html is copied into the image as a static snapshot. The running container uses that snapshot, not your live source code. Editing the source file on your host does not change the files inside the already-running container.

Why this is a feature in cloud computing:

1. **Consistency across environments** – The exact same image runs on a developer's laptop, a test server, and in production. No surprises caused by "someone forgot to update a file on the server."
2. **Reproducibility** – If a bug appears, you can rerun the exact same image to debug it. You don't have to guess what changes were made to a running server over time.
3. **Simplified rollbacks** – If a new version has a problem,

you simply redeploy the old image. You never "patch" a live container by editing files inside it.

4. **Scaling and automation** – When traffic increases, cloud platforms spin up dozens of identical containers from the same image. All of them behave exactly the same because they are built from an immutable artifact.
5. **Security and auditability** – No one can secretly change code inside a running container. Any change must go through the build pipeline, leaving an audit trail.